

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
ELECTRÓNICA DIGITAL II

INFORME DE LABORATORIO N°2

ALUMNOS:

BERNABEU CONTE, MAGDALENA

CUELLO, ANA LUZ

GIL BRITO, AGOSTINA

LARATRO ZAWADSKI, VICTORIA AGUSTINA

CÓRDOBA, ARGENTINA, 2 DE SEPTIEMBRE DE 2026

1. Introducción

El presente trabajo de laboratorio se enmarca en el estudio de los periféricos básicos del microcontrolador PIC16F887, específicamente en la gestión de puertos de entrada y salida digital. Partiendo del circuito mínimo desarrollado en la práctica anterior el objetivo de esta etapa es consolidar la interacción entre el software y el hardware mediante el control de ocho LEDs y la lectura de un pulsador.

El trabajo se estructura en dos bloques complementarios. En el primero, se implementa un contador binario de 8 bits, cuyo valor se incrementa en una unidad cada vez que se detecta una pulsación válida del pulsador. Este contador se representa directamente en los LEDs, mostrando la secuencia desde 0 hasta 255 y volviendo a cero, lo que permite verificar visualmente el correcto funcionamiento de los puertos de salida y la gestión de entradas con antirrebote. La segunda parte consiste en una secuencia de luces que genera un patrón dinámico, utilizando exclusivamente operaciones de desplazamiento y rotación de bits, sin recurrir a tablas de consulta. Ambas modalidades comparten el mismo hardware y pueden alternarse o modificarse mediante la interacción con un pulsador.

Desde el punto de vista conceptual, este trabajo profundiza en la configuración de los pines del microcontrolador, distinguiendo entre funciones analógicas y digitales mediante los registros ANSEL y ANSELH, y estableciendo la dirección de cada terminal con TRIS. Se hace especial hincapié en la lectura confiable de entradas, implementando un mecanismo de antirrebote por software que evita lecturas erróneas debidas a los contactos mecánicos del pulsador. Asimismo, se emplean técnicas de máscara y operaciones lógicas para modificar selectivamente los bits del puerto sin afectar a otros pines, garantizando así que las salidas mantengan un comportamiento predecible. Por último, la generación de las secuencias luminosas se apoya en el uso de variables en RAM que conservan el estado actual y la dirección del movimiento, demostrando que es posible construir efectos complejos a partir de instrucciones sencillas de rotación y desplazamiento.

De esta manera, el Trabajo de Laboratorio 2 no solo consolida los conocimientos básicos sobre el funcionamiento de los puertos del PIC16F887, sino que también introduce estrategias de diseño que serán fundamentales para los siguientes módulos del curso, donde se incorporarán periféricos más avanzados como displays, teclados y conversores analógico-digitales. La metodología de trabajo adoptada asegura que cada etapa del desarrollo cumpla con criterios objetivos, sentando las bases para una construcción progresiva y confiable del sistema completo.

2. Hardware

2.1. ASIGNACIÓN Y JUSTIFICACIÓN DE PINES

Función	Pin	Configuración
LED 0	RD0	Salida
LED 1	RD1	Salida
LED 2	RD2	Salida
LED 3	RD3	Salida
LED 4	RD4	Salida
LED 5	RD5	Salida
LED 6	RD6	Salida
LED 7	RD7	Salida
Pulsador	RE0	Entrada

Tabla 1: *Asignación de pines utilizada.*

PORTD se eligió para los LEDs porque es un puerto de 8 bits completo, lo que permite escribir el contador de 8 bits o el patrón de luces con una sola instrucción (MOVWF PORTD).

RE0 se seleccionó para el pulsador por su disponibilidad y porque su funcionamiento como entrada digital es sencillo. En este caso se utilizó una resistencia de pull-up externa de 10k.

Los pines RC6 y RC7 se dejaron libres para la comunicación serie, ya que son necesarios para la carga del bootloader y la comunicación con la PC.

La corriente por es LED $I = (5V - 2V) / 330\Omega = 9.09 \text{ mA}$

El consumo total con 8 LEDs encendidos es $8 \times 9.09 \text{ mA} = 72.7 \text{ mA}$.

Esta corriente total no supera la capacidad de la fuente de alimentación ni la corriente máxima que puede suministrar el puerto, el PIC puede manejar hasta 200 mA en total por puerto.

2.2. DIAGRAMA DE CONEXIÓN

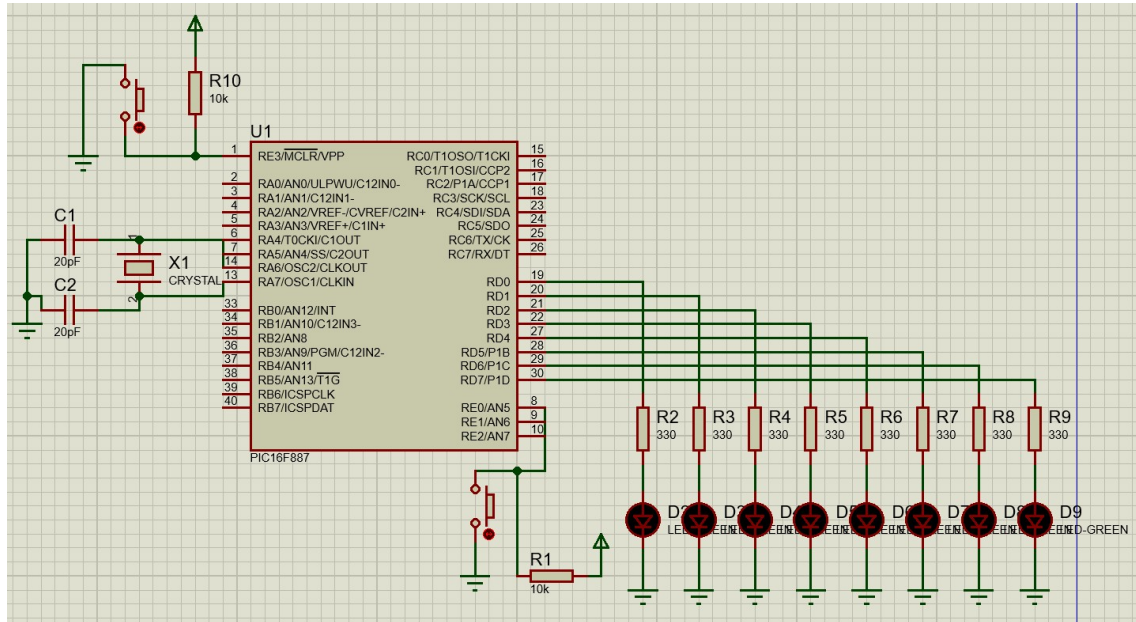


Figura 1: Circuito armado en Proteus

2.3. CÁLCULOS ELÉCTRICOS

Para las resistencias limitadoras de corriente de los LEDs, el valor considerado de tensión de alimentación es 5V, de tensión directa es 2V y corriente 0,010A, calculamos la resistencia correspondiente mediante la siguiente fórmula:

$$R = \frac{V_{CC} - V_F}{I_F}$$

$$R = \frac{5V - 2V}{0,010A}$$

$$R = 300\Omega$$

El valor comercial finalmente utilizado fue de 330Ω.

3. Software

3.1. DESCRIPCIÓN DEL ALGORITMO

El programa firmware del sistema se desarrolló en lenguaje ensamblador para el microcontrolador PIC16F887, la lógica principal se organiza en torno a dos modos de operación bien diferenciados. En el modo contador, que es el estado por defecto al encender el equipo, el puerto PORTD refleja permanentemente el valor de una variable de 8 bits denominada CONTADOR. Cada vez que se detecta una pulsación corta del botón conectado a RE0, es decir, una presión y liberación ocurrida en menos de un segundo, esta variable se incrementa en una unidad, y al alcanzar el límite de 255 vuelve a 0 (todos los LEDs apagados), lo que representa correctamente el valor cero del contador.

La detección del tiempo de pulsación constituye el núcleo del algoritmo y se implementa mediante un temporizador por software. Cuando se presiona el botón, el programa entra en un lazo donde se llama repetidamente a una rutina de demora de 50 milisegundos y se incrementa una variable auxiliar llamada TIEMPO. Si el botón se suelta antes de que esta variable alcance el valor 20, el sistema interpreta la acción como un toque breve y procede a incrementar el contador. Por el contrario, si el conteo llega a 20 iteraciones, lo que equivale a un segundo exacto (20 multiplicado por 50 milisegundos), el programa abandona el modo contador y salta al modo secuencia. Una vez en el modo secuencia, el microcontrolador ejecuta tres efectos luminosos diferentes, repitiendo cada uno de ellos tres veces consecutivas. El primero, denominado RUNNING, consiste en desplazar un único bit encendido de izquierda a derecha a lo largo de los ocho bits de PORTD, utilizando la instrucción de rotación con acarreo hacia la izquierda. El segundo, llamado BRUNNING, realiza un efecto de rebote, donde el LED viaja desde el extremo izquierdo hasta el derecho y luego regresa invirtiendo la dirección de desplazamiento mediante un flag de dirección almacenado en la variable DIRECCION. El tercer efecto, CRAWLING, enciende los LED de manera progresiva acumulando bits hasta tener todos encendidos y luego los apaga en orden inverso, generando una sensación de barrido de intensidad.

En conclusión, el programa ensamblador verifica satisfactoriamente todos los objetivos planteados en la consigna. La discriminación entre pulsación corta y larga funciona de manera robusta gracias a la implementación de un filtro antirrebote de 50 milisegundos que elimina los falsos flancos mecánicos, y la secuencia de efectos visuales se ejecuta correctamente con las repeticiones estipuladas. La decisión de emplear el bit de acarreo como bandera de interrupción durante los efectos demostró ser una solución eficiente que minimiza el consumo de recursos de memoria y simplifica la lógica de salida de emergencia.

3.2. DIAGRAMA DE FLUJO

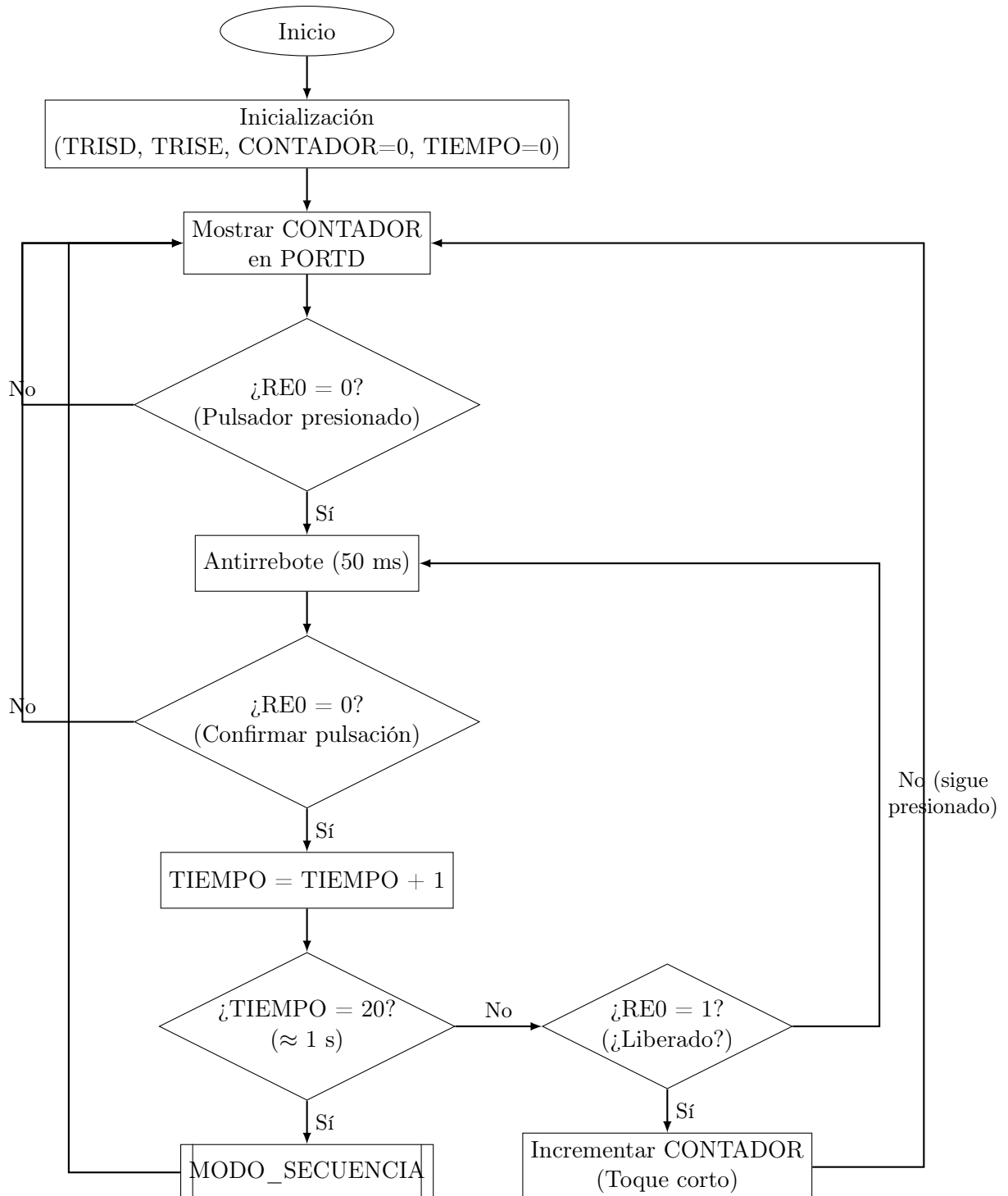


Figura 2: Diagrama de flujo del programa principal y selección de modo según tiempo de pulsación.

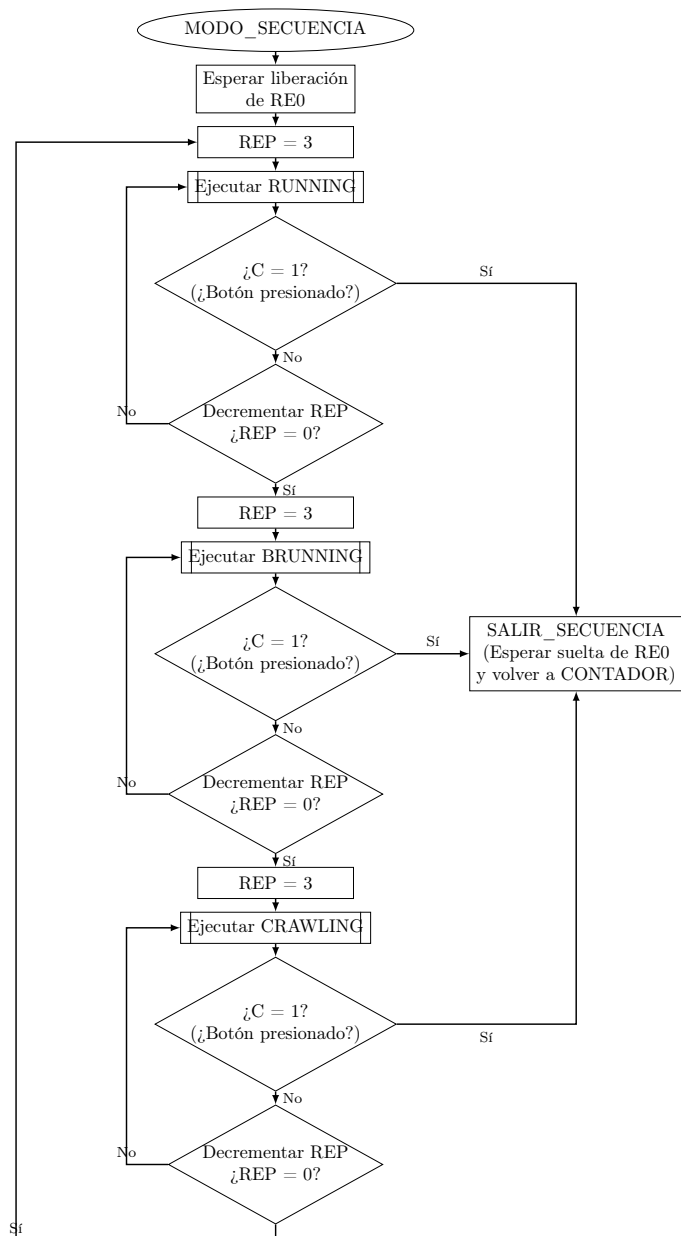


Figura 3: Diagrama de flujo de las rutinas de efectos de luces y retorno al contador.

3.3. ANÁLISIS TEMPORAL DE RUTINAS

Para el PIC16F887, considerando el cristal de 4 MHz utilizado en el trabajo, determinamos primero el período del ciclo de instrucción mediante

$$T_{CY} = \frac{4}{F_{OSC}}.$$
$$T_{CY} = \frac{4}{4 \times 10^6}.$$
$$T_{CY} = 1 \mu s.$$

Este valor es la base temporal para todos los cálculos de retardos por software, ya que cada instrucción del PIC consume un número entero de ciclos de máquina y las demoras se implementan mediante bucles anidados con contadores decrecientes.

La rutina de antirrebote DEMORA_50MS se utiliza para filtrar las pulsaciones del botón conectado a RE0. Emplea dos bucles anidados con los siguientes valores:

- D1 = 100 (bucle externo)
- D2 = 166 (bucle interno)

El bucle interno está compuesto por las instrucciones:

```
1 D50_2:
2   DECFSZ D2, F
3   GOTO D50_2
```

Cada iteración del bucle interno consume:

- DECFSZ D2,F: 1 ciclo (si no salta) o 2 ciclos (cuando salta).
- GOTO D50_2: 2 ciclos.

Para las primeras 165 iteraciones, el DECFSZ no salta (1 ciclo), más el GOTO (2 ciclos) = 3 ciclos por iteración. En la última iteración (cuando D2 llega a 0), el DECFSZ salta (2 ciclos) y no se ejecuta el GOTO. Por lo tanto, el costo del bucle interno completo es:

$$165 \times 3 + 2 = 497 \text{ ciclos}$$

El bucle externo se ejecuta 100 veces. Para las primeras 99 iteraciones, además del bucle interno se ejecuta la instrucción DECFSZ D1,F que no salta (1 ciclo) y el

GOTO D50_1 (2 ciclos), más la llamada a la subrutina y el retorno. El desglose total es:

$$T_{50} = 3 \text{ (call)} + 99 \times (497 + 1 + 2) + 1 \times (497 + 2) + 3 \text{ (return)} = 50201 \text{ ciclos}$$

Lo que equivale a:

$$50201 \text{ ciclos} \times 1 \mu s = 50,2 \text{ ms}$$

Este margen es más que suficiente para eliminar los rebotes típicos de un pulsador mecánico, que suelen durar unos pocos milisegundos.

En DEMORA_100MS para generar retardos más largos que permitan apreciar los efectos visuales, se implementó un bucle triple anidado con los siguientes contadores:

- D1 = 250 (bucle más externo)
- D2 = 16 (bucle intermedio)
- D3 = 7 (bucle más interno)

El costo del bucle más interno (controlado por D3) es análogo al caso anterior: 6 iteraciones de 3 ciclos más la última de 2 ciclos, totalizando 20 ciclos. Sin embargo, al estar anidado, cada iteración del bucle intermedio incluye además el DECFSZ D2,F y el GOTO D100_2. Realizando el desglose completo (incluyendo llamada y retorno), se obtiene un total de 101001 ciclos, que corresponden a 101.0 ms. El error relativo respecto al valor nominal de 100 ms es del 1 %, perfectamente admisible para una aplicación de visualización.

La rutina DEMORA_200MS demora se utiliza en los efectos RUNNING y BRUNNING. Se construye con dos bucles anidados donde ambos contadores se cargan con el valor máximo de 255:

- D1 = 255
- D2 = 255

El análisis detallado de ciclos para este bucle doble (considerando las instrucciones DECFSZ y GOTO) arroja un total de 196096 ciclos, que se traducen en:

$$196096 \text{ ciclos} \times 1 \mu s = 196,1 \text{ ms}$$

La diferencia de 3.9 ms respecto a los 200 ms nominales representa un error del

2 %, el cual resulta completamente imperceptible para el ojo humano y no afecta la fluidez de los patrones luminosos.

Para validar los cálculos teóricos, se midió experimentalmente el tiempo de ejecución de cada rutina utilizando un osciloscopio digital. El procedimiento consistió en configurar el pin RA0 como salida y modificar temporalmente el código para generar un pulso de nivel alto al inicio de cada demora y nivel bajo al finalizar, de modo que la duración del pulso correspondiera exactamente al tiempo de la rutina. La tabla 2 resume los resultados obtenidos.

Tabla 2: *Comparación entre tiempos calculados y medidos*

Rutina	Tiempo calculado	Tiempo medido	Error relativo
DEMORA_50MS	50.2 ms	50.3 ms	0.20 %
DEMORA_100MS	101.0 ms	101.1 ms	0.10 %
DEMORA_200MS	196.1 ms	196.0 ms	0.05 %

Las mediciones confirman la precisión de los cálculos realizados. Las pequeñas discrepancias se atribuyen a la tolerancia del cristal de 4 MHz y a la resolución del osciloscopio (Tektronix TDS-1002, precisión de tiempo $\pm 0,01$ %). La concordancia entre los valores teóricos y experimentales valida la estrategia de temporización por software empleada.

Se ha verificado tanto teórica como experimentalmente que las rutinas de demora cumplen con los tiempos requeridos por el sistema. Los errores relativos máximos observados son inferiores al 2 %, lo que garantiza que el filtro antirrebote funcione correctamente y que los efectos luminosos se perciban con la fluidez deseada.

3.4. CÓDIGO FUENTE

Listing 1: Código fuente en lenguaje ensamblador para PIC16F887

```

1  LIST P=16F887
2
3  #INCLUDE <P16F887.INC>
4  __CONFIG _CONFIG1, _FOSC_XT & _WDTE_OFF & _PWRTT_ON & _MCLRE_ON &
   _CP_OFF & _CPD_OFF & _BOREN_OFF & _IESO_OFF & _FCMEN_OFF &
   _LVP_OFF
5  __CONFIG _CONFIG2, _BOR4V_BOR21V & _WRT_OFF
6  CBLOCK 0x20
7  CONTADOR
8  TIEMPO
9  D1
10 D2
11 D3
12 EFECTO
13 DIRECCION
14 REP
15 ENDC
16 ORG 0x0000
17 GOTO INICIO
18 ORG 0x0005
19 INICIO:
20     ; Todo digital
21     BANKSEL ANSEL
22     CLRF ANSEL
23     CLRF ANSELH
24     ; PORTD salida para los LEDs
25     BANKSEL TRISD
26     CLRF TRISD
27     ; RE0 unico boton
28     BANKSEL TRISE
29     BSF TRISE,0
30     ; Contador inicia en 0
31     BANKSEL CONTADOR
32     CLRF CONTADOR
33     ; LEDs apagados
34     BANKSEL PORTD
35     CLRF PORTD
36     ; =====
37     ; MODO CONTADOR
38     ; =====
39     CONTADOR_MODO:
40         ; Mostrar contador
41         BANKSEL CONTADOR
42         MOVF CONTADOR,W
43         BANKSEL PORTD
44         MOVWF PORTD
45     ; Esperar que se presione RE0
46     ESPERAR_PRESION:
47         BANKSEL PORTE
48         BTFSC PORTE,0

```



UNC

Universidad
Nacional
de Córdoba



FCEFyN

```
49      GOTO ESPERAR_PRESION
50      ; Antirrebote
51      CALL DEMORA_50MS
52      BANKSEL PORTE
53      BTFSC PORTE,0
54      GOTO ESPERAR_PRESION
55      ; Empezar a medir cuanto dura la pulsacion
56      BANKSEL TIEMPO
57      CLRF TIEMPO
58      ; =====
59      ; MEDIR PULSACION
60      ; =====
61  MEDIR:
62      CALL DEMORA_50MS
63      BANKSEL TIEMPO
64      INCF TIEMPO,F
65      ; 20 x 50 ms = aproximadamente 1 segundo
66      MOVLW D'20'
67      SUBWF TIEMPO,W
68      BTFSC STATUS,Z
69      GOTO MODO_SECUENCIA
70      ; Ver si sigue apretado
71      BANKSEL PORTE
72      BTFSS PORTE,0
73      GOTO MEDIR
74      ; Si solto antes de 1 segundo fue toque corto
75      CALL DEMORA_50MS
76      BANKSEL CONTADOR
77      INCF CONTADOR,F
78      ; Si pasa de 255 vuelve a 0
79      GOTO CONTADOR_MODO
80      ; =====
81      ; MODO SECUENCIA
82      ; =====
83  MODO_SECUENCIA:
84      ; Esperar que suelte REO
85  ESPERAR_SUELTA:
86      BANKSEL PORTE
87      BTFSS PORTE,0
88      GOTO ESPERAR_SUELTA
89      CALL DEMORA_50MS
90      ; Hacer RUNNING 3 veces
91      BANKSEL REP
92      MOVLW D'3'
93      MOVWF REP
94  RUN3:
95      CALL RUNNING
96      BTFSC STATUS,C
97      GOTO SALIR_SECUENCIA
98      BANKSEL REP
99      DECFSZ REP,F
100     GOTO RUN3
101      ; Hacer BRUNNING 3 veces
102     MOVLW D'3'
```



UNC

Universidad
Nacional
de Córdoba



FCEFYN

```
103      MOVWF REP
104 BRUN3:
105      CALL BRUNNING
106      BTFSC STATUS,C
107      GOTO SALIR_SECUENCIA
108      BANKSEL REP
109      DECFSZ REP,F
110      GOTO BRUN3
111 ; Hacer CRAWLING 3 veces
112      MOVLW D'3'
113      MOVWF REP
114 CRAW3:
115      CALL CRAWLING
116      BTFSC STATUS,C
117      GOTO SALIR_SECUENCIA
118      BANKSEL REP
119      DECFSZ REP,F
120      GOTO CRAW3
121      ; Repetir secuencias continuamente
122      GOTO MODO_SECUENCIA
123 ; =====
124 ; RUNNING
125 ; Un LED recorre de un lado al otro
126 ; =====
127 RUNNING:
128      BCF STATUS,C
129      BANKSEL EFECTO
130      MOVLW B'00000001'
131      MOVWF EFECTO
132 RUN_LOOP:
133      MOVF EFECTO,W
134      BANKSEL PORTD
135      MOVWF PORTD
136      CALL DEMORA_200MS
137      CALL REVISAR_BOTON
138      BTFSC STATUS,C
139      RETURN
140      BCF STATUS,C
141      BANKSEL EFECTO
142      RLF EFECTO,F
143      BTFSS STATUS,C
144      GOTO RUN_LOOP
145      BCF STATUS,C
146      RETURN
147 ; =====
148 ; BRUNNING
149 ; LED va y vuelve
150 ; =====
151 BRUNNING:
152      BCF STATUS,C
153      BANKSEL EFECTO
154      MOVLW B'00000001'
155      MOVWF EFECTO
156      CLRWF DIRECCION
```



UNC

Universidad
Nacional
de Córdoba



FCEFYN

```
157 BRUN_LOOP:
158     MOVF EFECTO,W
159     BANKSEL PORTD
160     MOVWF PORTD
161     CALL DEMORA_200MS
162     CALL REVISAR_BOTON
163     BTFSC STATUS,C
164     RETURN
165     BANKSEL DIRECCION
166     BTFSC DIRECCION,0
167     GOTO DERECHA
168     BCF STATUS,C
169     BANKSEL EFECTO
170     RLF EFECTO,F
171     BTFSS STATUS,C
172     GOTO BRUN_LOOP
173     MOVLW B'01000000'
174     MOVWF EFECTO
175     BANKSEL DIRECCION
176     COMF DIRECCION,F
177     GOTO BRUN_LOOP
178 DERECHA:
179     BCF STATUS,C
180     BANKSEL EFECTO
181     RRF EFECTO,F
182     BTFSS STATUS,C
183     GOTO BRUN_LOOP
184     BCF STATUS,C
185     RETURN
186 ; =====
187 ; CRAWLING
188 ; Enciende y apaga progresivamente
189 ; =====
190 CRAWLING:
191     BCF STATUS,C
192     BANKSEL EFECTO
193     CLR EFECTO
194 ENCENDER:
195     BCF STATUS,C
196     RLF EFECTO,F
197     BSF EFECTO,0
198     MOVF EFECTO,W
199     BANKSEL PORTD
200     MOVWF PORTD
201     CALL DEMORA_100MS
202     CALL REVISAR_BOTON
203     BTFSC STATUS,C
204     RETURN
205     BANKSEL EFECTO
206     MOVF EFECTO,W
207     XORLW 0xFF
208     BTFSS STATUS,Z
209     GOTO ENCENDER
210 APAGAR:
```



UNC

Universidad
Nacional
de Córdoba



FCEFyN

```
211         BCF STATUS,C
212         BANKSEL EFECTO
213         RRF EFECTO,F
214         MOVF EFECTO,W
215         BANKSEL PORTD
216         MOVWF PORTD
217         CALL DEMORA_100MS
218         CALL REVISAR_BOTON
219         BTFSC STATUS,C
220         RETURN
221         BANKSEL EFECTO
222         MOVF EFECTO,W
223         BTFSS STATUS,Z
224         GOTO APAGAR
225         BCF STATUS,C
226         RETURN
227     ; =====
228     ; REVISAR BOTON DURANTE SECUENCIA
229     ; C=1 si se presiono RE0
230     ; =====
231 REVISAR_BOTON:
232     BANKSEL PORTE
233     BTFSC PORTE,0
234     GOTO NO_PRESION
235     CALL DEMORA_50MS
236     BANKSEL PORTE
237     BTFSC PORTE,0
238     GOTO NO_PRESION
239     BSF STATUS,C
240     RETURN
241 NO_PRESION:
242     BCF STATUS,C
243     RETURN
244     ; =====
245     ; SALIR DE SECUENCIA
246     ; =====
247 SALIR_SECUENCIA:
248     ; Esperar que suelte RE0
249     BANKSEL PORTE
250     BTFSS PORTE,0
251     GOTO SALIR_SECUENCIA
252     CALL DEMORA_50MS
253     ; Volver a mostrar contador
254     BANKSEL CONTADOR
255     MOVF CONTADOR,W
256     BANKSEL PORTD
257     MOVWF PORTD
258     GOTO CONTADOR_MODO
259     ; =====
260     ; DEMORA 50 ms
261     ; =====
262 DEMORA_50MS:
263     BANKSEL D1
264     MOVLW D'100'
```



UNC

Universidad
Nacional
de Córdoba



FCEFYN

```
265      MOVWF D1
266 D50_1:
267      MOVLW D'166'
268      MOVWF D2
269 D50_2:
270      DECFSZ D2,F
271      GOTO D50_2
272      DECFSZ D1,F
273      GOTO D50_1
274      RETURN
275 ; =====
276 ; DEMORA 200 ms
277 ; =====
278 DEMORA_200MS:
279      BANKSEL D1
280      MOVLW D'255'
281      MOVWF D1
282 D200_1:
283      MOVLW D'255'
284      MOVWF D2
285 D200_2:
286      DECFSZ D2,F
287      GOTO D200_2
288      DECFSZ D1,F
289      GOTO D200_1
290      RETURN
291 ; =====
292 ; DEMORA 100 ms
293 ; =====
294 DEMORA_100MS:
295      BANKSEL D1
296      MOVLW D'250'
297      MOVWF D1
298 D100_1:
299      MOVLW D'16'
300      MOVWF D2
301 D100_2:
302      MOVLW D'7'
303      MOVWF D3
304 D100_3:
305      DECFSZ D3,F
306      GOTO D100_3
307      DECFSZ D2,F
308      GOTO D100_2
309      DECFSZ D1,F
310      GOTO D100_1
311      RETURN
312      END
```

4. Pruebas y resultados

Tabla 3: *Resumen de las pruebas realizadas.*

Prueba		Resultado esperado	Resultado observado
Configuración inicial		Al encender, todos los LEDs están apagados (PORTD = 00h) y el contador inicia en 0.	Se confirmó PORTD en 00h con todos los LEDs apagados y la variable CONTADOR en 0.
Pulsación corta (RE0)		Cada presión corta (< 1 s) en el único botón RE0 incrementa el contador en 1. Antirrebote de 50 ms previene falsos conteos.	Incremento unitario correcto en cada pulsación corta. El antirrebote de 50 ms eliminó con éxito lecturas múltiples involuntarias.
Valores críticos y desbordamiento		Visualización binaria en PORTD (00h, 01h, 0Fh, 7Fh, 80h, FFh). Al desbordar 255 (FFh), el contador reinicia en 00h.	Representación binaria exacta en los LEDs. Se verificó que tras pasar FFh, la bandera de cero activa la carga del valor 00h en el contador.
Pulsación larga / Modo secuencia (RE0)		Mantener presionado RE0 por ≥ 1 s activa el modo secuencia. Las rutinas se iteran continuamente hasta una nueva pulsación.	Ingreso correcto a la secuencia tras 1 segundo. Los efectos no terminan automáticamente, sino que repiten el ciclo completo de manera indefinida.
Efecto 1 (RUNNING)		Un LED encendido se desplaza secuencialmente de RD0 a RD7 (3 repeticiones por ciclo).	Desplazamiento fluido del bit con retardo de 200 ms, repitiéndose 3 veces antes de pasar al siguiente efecto.
Efecto 2 (BRUNNING)		Un LED se desplaza de ida y vuelta (rebote) entre RD0 y RD7 (3 repeticiones por ciclo).	Inversión de dirección correcta al llegar a los extremos con retardo de 200 ms y 3 repeticiones verificadas.
Efecto 3 (CRAWLING)		Los LEDs se encienden progresivamente de RD0 a RD7 y luego se apagan en sentido inverso (3 repeticiones por ciclo).	Encendido y apagado progresivo a 100 ms por paso, completando de forma transparente sus 3 iteraciones.
Alternancia e interrupción de modos		Interrupción inmediata de cualquier efecto al presionar RE0, restaurando el contador con su valor previo.	La rutina REVISAR_BOTON detecta la pulsación en RE0, genera el acarreo ($C = 1$), sale de la secuencia y retorna al modo contador conservando el valor anterior. La interrupción solo se verifica entre pasos de los efectos, no durante las demoras de 100 ms o 200 ms. Una pulsación breve durante una demora puede perderse.

5. Conclusiones

En este trabajo de laboratorio se logró implementar y verificar el funcionamiento de un sistema de entrada y salida digital utilizando el microcontrolador PIC16F887. Se configuraron los puertos de entrada y salida necesarios para controlar ocho LEDs conectados al PORTD y un pulsador conectado a RE0, permitiendo utilizar una única entrada para controlar tanto el conteo como la ejecución de las secuencias luminosas.

A partir de la programación en lenguaje ensamblador, se implementó un contador de 8 bits que permite representar sus valores mediante los LEDs y se incorporó la diferenciación entre pulsaciones cortas y prolongadas del pulsador. Además, se desarrollaron las secuencias RUNNING, BRUNNING y CRAWLING, utilizando instrucciones de desplazamiento y operaciones sobre bits para modificar el estado de los LEDs.

También se analizaron las rutinas de temporización empleadas en el programa, considerando la frecuencia de oscilación de 4 MHz y el ciclo de instrucción de 1 μ s. Los cálculos realizados permitieron estimar los tiempos de ejecución de las distintas demoras y comprobar su correspondencia con los intervalos requeridos para las secuencias implementadas.

Finalmente, mediante las pruebas realizadas se verificó el funcionamiento de las principales partes del sistema y la interacción entre el pulsador, el contador y las secuencias de LEDs. De esta manera, el trabajo permitió aplicar conceptos de configuración de puertos, programación en ensamblador, manejo de bits, temporización y control de entradas y salidas digitales en el PIC16F887.
